

Flow and Working of Voice Based Examination System Project

This document explains the complete architecture, working flow, folder structure, backend/frontend interaction, voice engine flow, and overall execution lifecycle of the Voice Based Examination System project.

1. Project Overview

The project is a Voice Based Examination System built using Flask architecture.

It supports:

- Admins
- Candidates
- Examiners
- Invigilators

Core technologies include:

- Python + Flask
- HTML/CSS/JavaScript
- Speech Recognition
- Whisper.cpp
- Database ORM models
- AI-assisted grading

2. High Level System Flow

```
run.py
↓
Flask Application Starts
↓
App Initialization
↓
Database + Middleware + Routes Loaded
↓
Frontend Templates Rendered
↓
JavaScript Voice Engine Starts
↓
User Voice Commands Processed
↓
Backend Routes Receive Requests
↓
Services Execute Business Logic
↓
Database Stores Answers
↓
AI/Grading Services Process Results
```

3. Root Files

run.py:

- Main entry point
- Starts Flask server

requirements.txt:

- Stores dependencies

.env.example:

- Environment configuration

README.md:

- Documentation and setup guide

4. App Initialization

app/__init__.py initializes:

- Flask app
- Database
- Routes
- Middleware
- Configurations

This acts as the application bootstrapper.

5. Constants Layer

constants/ folder stores reusable fixed values.

roles.py:

- ADMIN
- CANDIDATE
- EXAMINER
- INVIGILATOR

status.py:

- EXAM_STARTED
- SUBMITTED
- COMPLETED

6. Database Layer

database/models.py defines ORM models such as:

- User
- Exam

- Question
- Answer

seed.py and seed_demo_exam.py populate default data and demo exams.

7. Middleware Layer

Middleware processes requests before routes.

auth_middleware.py:

- Login validation
- Permission checks

logging_middleware.py:

- Request logging
- Error logging

request_middleware.py:

- Request processing
- Metadata handling

8. Routes Layer

Routes act as API endpoints.

auth_routes.py:

- Login
- Logout
- Registration

candidate_routes.py:

- Start exam
- Submit answers
- View results

admin_routes.py:

- Manage users
- Manage exams

examiner_routes.py:

- Grading operations

invigilator_routes.py:

- Monitoring operations

9. Services Layer

Services contain the actual business logic.

Important services:

- auth_service.py
- exam_service.py
- candidate_service.py
- speech_service.py
- grading_service.py
- ai_service.py
- crypto_service.py

10. Frontend Static Files

static/css:

- UI styling
- Dashboard layouts
- Responsive design

static/js:

- Frontend interactions
- Voice engine
- API calls

11. Voice Engine Architecture

The most innovative part of the project is:

static/js/voice/

Modules include:

- recognition.js
- transcript.js
- normalization.js
- commands.js
- autosave.js
- recovery.js
- anti_cheat.js
- timer.js
- lifecycle.js

12. Voice Processing Flow

Microphone Input

↓

recognition.js

↓

transcript.js

↓

normalization.js
↓
command_matcher.js
↓
commands.js
↓
voice/api.js
↓
Backend Routes
↓
Database Save

13. Voice Module Responsibilities

main.js:

- Starts voice engine

recognition.js:

- Speech-to-text

transcript.js:

- Transcript management

normalization.js:

- Cleans spoken text

command_matcher.js:

- Detects commands

commands.js:

- Executes commands

autosave.js:

- Periodic answer saving

recovery.js:

- Crash recovery

tts.js:

- Text-to-Speech

state.js:

- Frontend state manager

14. Templates Layer

templates/ contains Flask-rendered HTML pages.

Folders:

- admin/
- candidate/

- examiner/
- invigilator/
- shared/

Example flow:

candidate_routes.py

↓

render_template("student_dashboard.html")

↓

Frontend loads

↓

Voice JS attached

15. Utilities and Validators

utils/:

- decorators
- helper functions
- formatting utilities

validators/:

- exam validation
- question validation
- user validation

16. Whisper.cpp Integration

whisper.cpp-master/ is a third-party speech engine.

Purpose:

Audio

↓

Whisper.cpp

↓

Speech-to-text

Used for:

- Offline transcription
- Voice processing
- Speech recognition

17. Complete Exam Lifecycle

STEP 1:

Student opens dashboard

STEP 2:

Voice engine initializes

STEP 3:
Student speaks answer

STEP 4:
Speech converted to text

STEP 5:
Commands detected

STEP 6:
API request sent

STEP 7:
Backend services process request

STEP 8:
Database stores answer

STEP 9:
Autosave periodically saves progress

STEP 10:
Submission evaluated and graded

18. Architecture Summary

The project follows layered architecture:

- Routes → Request handling
- Services → Business logic
- Models → Database layer
- Templates → UI rendering
- Voice Modules → Speech engine
- Middleware → Security and logging
- Validators → Input validation

19. Most Important Files

Key files:

- run.py
- app/__init__.py
- models.py
- exam_service.py
- speech_service.py
- recognition.js
- main.js
- autosave.js
- candidate_routes.py
- auth_service.py

20. Strongest Part of the Project

The strongest component is:
`static/js/voice/`

This module demonstrates:

- Speech recognition
- Voice lifecycle management
- Command processing
- Autosave
- Recovery system
- Anti-cheat concepts
- Transcript normalization

This is the main innovation layer of the project.